Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To train an agent based on the environment to find the shortest path using Reinforcement Learning	
Experiment No: 14	**Date:**	**Enrolment No: 92301733050**

Aim: To train an agent based on the environment to find the shortest path using Reinforcement Learning

IDE: Google Colab

Theory:

Reinforcement learning is an area of Machine Learning. It is about taking suitable action to maximize reward in a particular situation. It is employed by various software and machines to find the best possible behavior or path it should take in a specific situation. Reinforcement learning differs from supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task. In the absence of a training dataset, it is bound to learn from its experience.

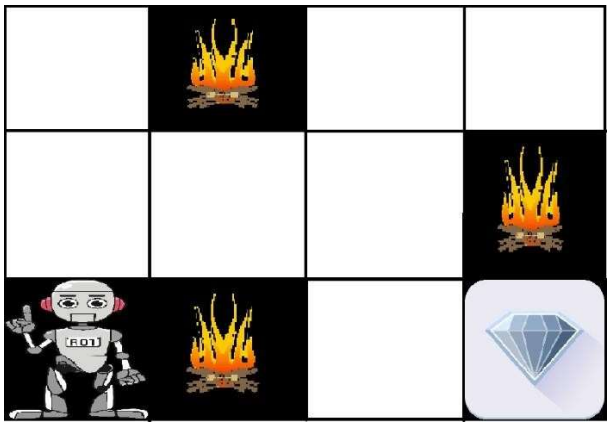
Reinforcement Learning (RL) is the science of decision making. It is about learning the optimal behavior in an environment to obtain maximum reward. In RL, the data is accumulated from machine learning systems that use a trial-and-error method. Data is not part of the input that we would find in supervised or unsupervised machine learning.

Reinforcement learning uses algorithms that learn from outcomes and decide which action to take next. After each action, the algorithm receives feedback that helps it determine whether the choice it made was correct, neutral or incorrect. It is a good technique to use for automated systems that have to make a lot of small decisions without human guidance.

Reinforcement learning is an autonomous, self-teaching system that essentially learns by trial and error. It performs actions with the aim of maximizing rewards, or in other words, it is learning by doing in order to achieve the best outcomes.

Example:

The problem is as follows: We have an agent and a reward, with many hurdles in between. The agent is supposed to find the best possible path to reach the reward. The following problem explains the problem more easily.



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To train an agent based on the environment to find the shortest path using Reinforcement Learning	
Experiment No: 14	**Date:**	**Enrolment No: 92301733050**

The above image shows the robot, diamond, and fire. The goal of the robot is to get the reward that is the diamond and avoid the hurdles that are fired. The robot learns by trying all the possible paths and then choosing the path which gives him the reward with the least hurdles. Each right step will give the robot a reward and each wrong step will subtract the reward of the robot. The total reward will be calculated when it reaches the final reward that is the diamond.

Main points in Reinforcement learning –

- Input: The input should be an initial state from which the model will start
- Output: There are many possible outputs as there are a variety of solutions to a particular problem
- Training: The training is based upon the input, The model will return a state and the user will decide to reward or punish the model based on its output.
- The model keeps continues to learn.
- The best solution is decided based on the maximum reward.

Types of Reinforcement:

There are two types of Reinforcement:

1. **Positive:** Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior.
   Advantages of reinforcement learning are:
   - Maximizes Performance
   - Sustain Change for a long period of time
   - Too much Reinforcement can lead to an overload of states which can diminish the results
2. **Negative:** Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided. Advantages of reinforcement learning:
   - Increases Behavior
   - Provide defiance to a minimum standard of performance
   - It Only provides enough to meet up the minimum behavior Reinforcement learning elements are as follows:

1. Policy
2. Reward function
3. Value function
4. Model of the environment

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To train an agent based on the environment to find the shortest path using Reinforcement Learning	
Experiment No: 14	**Date:**	**Enrolment No: 92301733050**

Policy: Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.
Reward function: Reward function is used to define a goal in a reinforcement learning problem.A reward function is a function that provides a numerical score based on the state of the environment
Value function: Value functions specify what is good in the long run. The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state. **Model of the environment:** Models are used for planning.

Credit assignment problem: Reinforcement learning algorithms learn to generate an internal value for the intermediate states as to how good they are in leading to the goal. The learning decision maker is called the agent. The agent interacts with the environment that includes everything outside the agent.
The agent has sensors to decide on its state in the environment and takes action that modifies its state.

The reinforcement learning problem model is an agent continuously interacting with an environment. The agent and the environment interact in a sequence of time steps. At each time step t, the agent receives the state of the environment and a scalar numerical reward for the previous action, and then the agent then selects an action.

Reinforcement learning is a technique for solving Markov decision problems. Reinforcement learning uses a formal framework defining the interaction between a learning agent and its environment in terms of states, actions, and rewards. This framework is intended to be a simple way of representing essential features of the artificial intelligence problem.

Various Practical Applications of Reinforcement Learning –

- RL can be used in robotics for industrial automation.
- RL can be used in machine learning and data processing
- RL can be used to create training systems that provide custom instruction and materials according to the requirement of students.

Application of Reinforcement Learnings

1. Robotics: Robots with pre-programmed behavior are useful in structured environments, such as the assembly line of an automobile manufacturing plant, where the task is repetitive in nature.
2. A master chess player makes a move. The choice is informed both by planning, anticipating possible replies and counter replies.
3. An adaptive controller adjusts parameters of a petroleum refinery's operation in real time.

RL can be used in large environments in the following situations:

1. A model of the environment is known, but an analytic solution is not available;

2. Only a simulation model of the environment is given (the subject of simulation-based optimization)
3. The only way to collect information about the environment is to interact with it.

Advantages and Disadvantages of Reinforcement Learning

Advantages of Reinforcement learning

1. Reinforcement learning can be used to solve very complex problems that cannot be solved by conventional techniques.
2. The model can correct the errors that occurred during the training process.
3. In RL, training data is obtained via the direct interaction of the agent with the environment
4. Reinforcement learning can handle environments that are non-deterministic, meaning that the outcomes of actions are not always predictable. This is useful in real-world applications where the environment may change over time or is uncertain.
5. Reinforcement learning can be used to solve a wide range of problems, including those that involve decision making, control, and optimization.
6. Reinforcement learning is a flexible approach that can be combined with other machine learning techniques, such as deep learning, to improve performance.

Disadvantages of Reinforcement learning

1. Reinforcement learning is not preferable to use for solving simple problems.
2. Reinforcement learning needs a lot of data and a lot of computation
3. Reinforcement learning is highly dependent on the quality of the reward function. If the reward function is poorly designed, the agent may not learn the desired behavior.
4. Reinforcement learning can be difficult to debug and interpret. It is not always clear why the agent is behaving in a certain way, which can make it difficult to diagnose and fix problems.

Pre Lab Exercise:

1. How does reinforcement learning differ from supervised and unsupervised learning?

Reinforcement Learning (RL) differs from supervised and unsupervised learning mainly in how learning happens:

- In **Supervised Learning**, the model is trained using labeled data (input + correct output). The model learns by comparing predictions with known answers.
- In **Unsupervised Learning**, there are no labels. The model finds hidden patterns or structures in data.

- In **Reinforcement Learning**, there is **no predefined dataset or correct answer**. The agent learns by interacting with an environment using **trial and error**, receiving rewards or penalties for actions.

2. What are the key components of a reinforcement learning system and explain them.

- **Agent**
  The learner or decision-maker that interacts with the environment.
- **Environment**
  Everything outside the agent. It responds to the agent's actions and provides feedback.
- **State (S)**
  The current situation or position of the agent in the environment.
- **Action (A)**
  The set of all possible moves the agent can take.
- **Reward (R)**
  A numerical value given after each action. It indicates whether the action was good or bad.
- **Policy (π)**
  A strategy that defines what action the agent should take in a given state.
- **Value Function (V)**
  Estimates how good a state is in terms of future rewards.
- **Model (optional)**
  Represents how the environment behaves (used in planning).

3. Describe the difference between exploration and exploitation in reinforcement learning.

- **Exploration**
  The agent tries new actions to discover better rewards.
  Example: Trying a new path in a maze.
- **Exploitation**
  The agent uses known information to maximize reward.
  Example: Choosing the best-known path.

4. What is the purpose of Q-Learning in reinforcement learning.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To train an agent based on the environment to find the shortest path using Reinforcement Learning	
Experiment No: 14	**Date:**	**Enrolment No: 92301733050**

Q-Learning is a **model-free RL algorithm** used to find the **optimal action-selection policy**.

- It learns a **Q-value (Quality value)** for each state-action pair.
- Q-value represents the expected future reward.
- The agent updates Q-values using the **Bellman Equation**.

Program (Code):

To be attached with

```
import numpy as np
import pylab as pl
import networkx as nx
edges = [(0, 1), (1, 5), (5, 6), (5, 4), (1, 2),
   (1, 3), (9, 10), (2, 4), (0, 6), (6, 7),
   (8, 9), (7, 8), (1, 7), (3, 9),(10,8),(10,10)]

goal = 10
G = nx.Graph()
G.add_edges_from(edges)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G,  pos)
nx.draw_networkx_edges(G, pos)
nx.draw_networkx_labels(G, pos)
pl.show()
MATRIX_SIZE = 11
R = np.matrix(np.ones(shape =(MATRIX_SIZE, MATRIX_SIZE)))
R *= -1

for point in edges:
  print(point)
  if point[1] == goal:  #GOAL IS 10
    R[point] = 100
  else:
    R[point] = 0

  if point[0] == goal:
    R[point[::-1]] = 100    #SEQUENCE [START: STOP: STEP] shows the output in reverse order.  (9->10) gets
reward but (10,9) doesn't
```

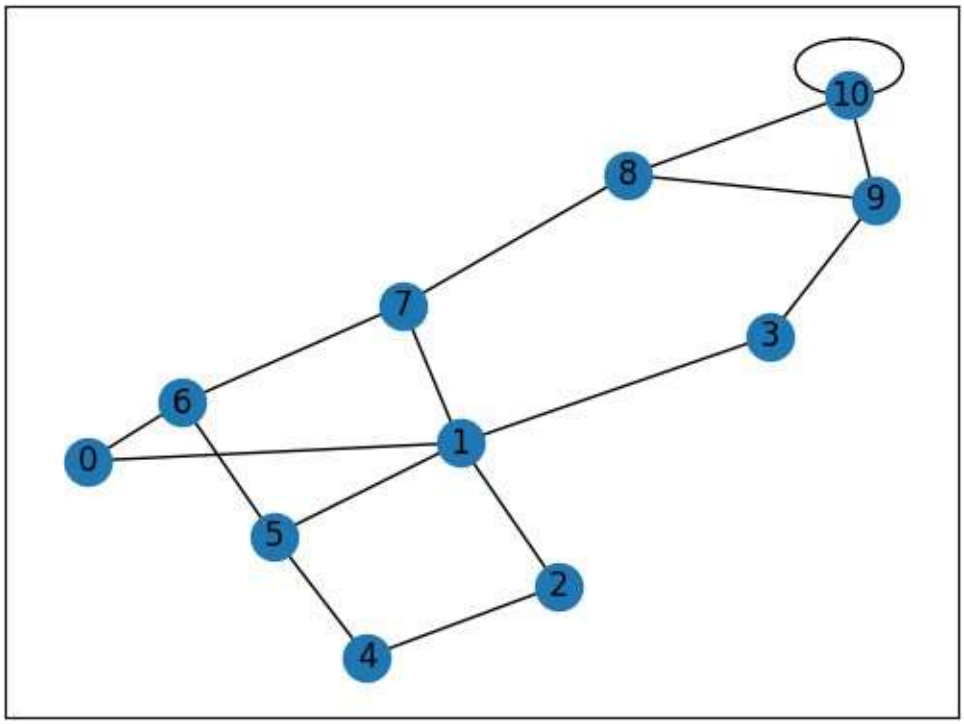
```
  else:
    R[point[::-1]]= 0   #REVERSE ALSO SAME POINT AS O
   # reverse of point

R[goal, goal]= 100
print(R)
# add goal point round trip
 Q = np.matrix(np.zeros([MATRIX_SIZE, MATRIX_SIZE]))

gamma = 0.75
# learning parameter
initial_state = 1

# Determines the available actions for a given state
def available_actions(state):
  current_state_row = R[state, ]
  available_action = np.where(current_state_row >= 0)[1]
  return available_action

available_action = available_actions(initial_state)
print("available actions: ", available_action)
# Chooses one of the available actions at random
def sample_next_action(available_actions_range):
  next_action = int(np.random.choice(available_action, 1))
  return next_action

action = sample_next_action(available_action)
print("next actions: ", action)

def update(current_state, action, gamma):
  max_index = np.where(Q[action, ] == np.max(Q[action, ]))[1]
  if max_index.shape[0] > 1:
    print("max_index: ",max_index)
    max_index = int(np.random.choice(max_index, size = 1))   #it will give output in aarray format
  else:
    print("max_index: ",max_index)
    max_index = int(max_index)   #It will be in the array format
```

```
  max_value = Q[action, max_index]
  Q[current_state, action] = R[current_state, action] + gamma * max_value
  if (np.max(Q) > 0):
    return(np.sum(Q / np.max(Q)*100))
  else:
    return (0)
# Updates the Q-Matrix according to the path chosen
print("Before Q matrix: ")
print(Q)
update(initial_state, action, gamma)
print("After Q matrix: ")
print(Q)
scores = []
for i in range(1000):
  current_state = np.random.randint(0, int(Q.shape[0]))
  available_action = available_actions(current_state)
  action = sample_next_action(available_action)
  score = update(current_state, action, gamma)
  scores.append(score)

print("Trained Q matrix:")
print(Q / np.max(Q)*100)
# You can uncomment the above two lines to view the trained Q matrix

pl.plot(scores)
pl.xlabel('No of iterations')
pl.ylabel('Reward gained')
pl.show()
current_state = 2
steps = [current_state]

while current_state != 10:

  next_step_index = np.where(Q[current_state, ] == np.max(Q[current_state, ]))[1]
  if next_step_index.shape[0] > 1:
    next_step_index = int(np.random.choice(next_step_index, size = 1))
  else:
```

```
    next_step_index = int(next_step_index)
  steps.append(next_step_index)
  current_state = next_step_index

print("Most efficient path:")
print(steps)
```

Results:

To be attached with

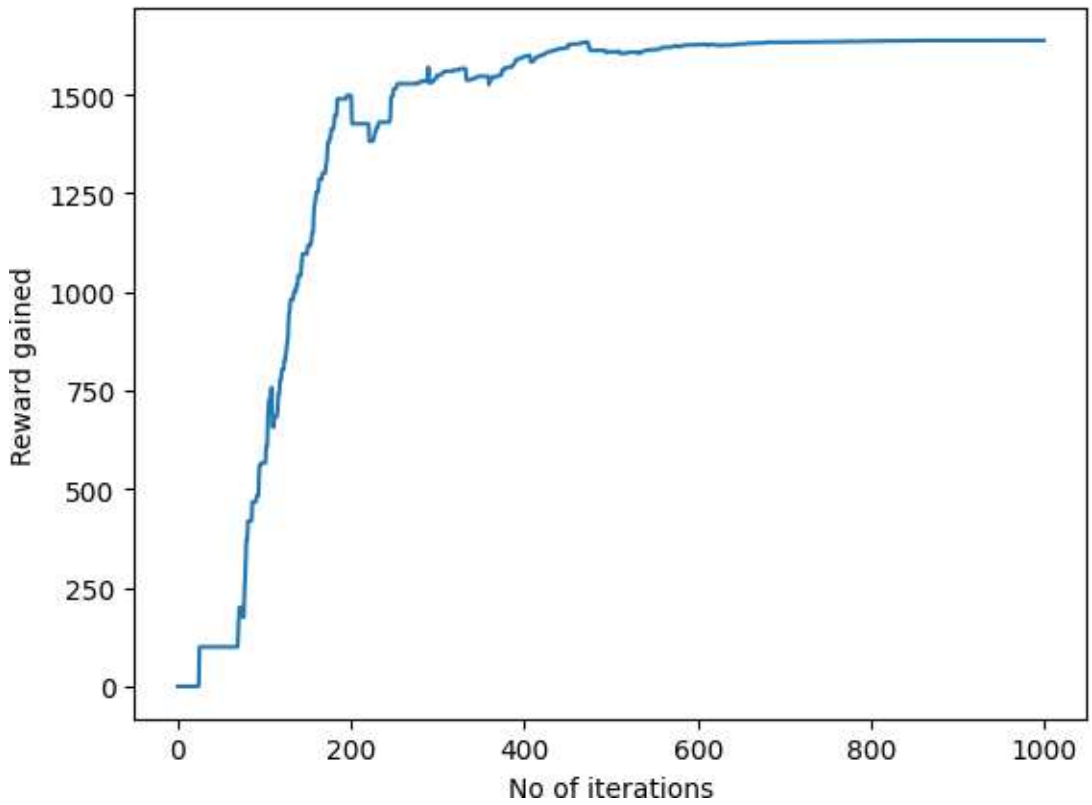


Trained Q matrix:

```
[[  0.          42.18314412   0.          0.          0.
    0.          42.18314412   0.          0.          0.
    0.        ]
 [ 31.63296296   0.          31.63296296  56.24084349   0.
   31.63296296   0.          56.24419216   0.          0.
    0.        ]
```

```
[  0.          42.18063262   0.          0.          23.72801857
   0.          0.          0.          0.          0.
   0.        ]
[  0.          42.18314412   0.          0.          0.
   0.          0.          0.          0.          74.98779132
   0.        ]
[  0.          0.          31.63296296   0.          0.
  31.63735809   0.          0.          0.          0.
   0.        ]
[  0.          42.18063262   0.          0.          23.72801857
   0.          42.18314412   0.          0.          0.
   0.        ]
[ 31.63735809   0.          0.          0.          0.
  31.63735809   0.          56.25         0.          0.
   0.        ]
[  0.          42.18314412   0.          0.          0.
   0.          42.18314412   0.          75.          0.
   0.        ]
[  0.          0.          0.          0.          0.
   0.          0.          56.25         0.          74.98779132
 100.        ]
[  0.          0.          0.          56.24084349   0.
   0.          0.          0.          74.99560488   0.
  99.98   72176]
[  0.          0.          0.          0.          0.
   0.          0.          0.          75.          74.98779132
  99.9974885 ]]
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To train an agent based on the environment to find the shortest path using Reinforcement Learning	
Experiment No: 14	**Date:**	**Enrolment No: 92301733050**



Most efficient path:
[2, 1, 7, 8, 10]

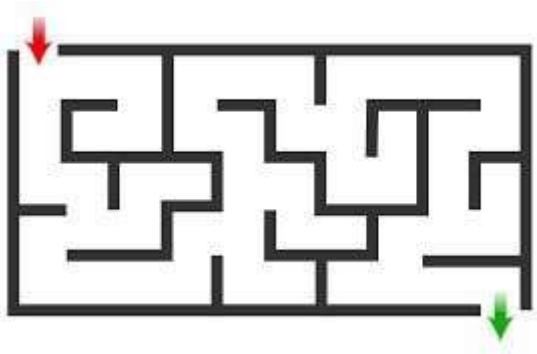
Observation:

- The agent initially explores the maze randomly and receives rewards or penalties.
- Over multiple episodes, the agent updates its Q-values and improves its decisions.
- The agent gradually avoids obstacles (fire) and moves toward the goal (diamond).
- The total reward increases over time, indicating learning.
- Eventually, the agent converges to the **shortest and safest path**.
- Learning speed depends on:
  - Learning rate (α)
  - Discount factor (γ)
  - Exploration rate (ε)

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To train an agent based on the environment to find the shortest path using Reinforcement Learning	
Experiment No: 14	**Date:**	**Enrolment No: 92301733050**

Post Lab Exercise:

Implement the reinforcement learning to train an agent to travel through the maze given in the image below. Also, explain the step-by-step procedure to train the agent and get the output sequence path for the same.



Step-1: Problem Understanding

- The maze is an **environment**
- The agent starts at the **red arrow (start)**
- The goal is the **green arrow (end)**
- The agent must learn the best path through **trial and error**

Step-2: Model the Problem as RL

- **State (S)**
  - Each cell in the maze = one state
  - Represented as coordinates (row, col)
- **Actions (A)**
  - At each state, the agent can:
  - Up ↑
  - Down ↓
  - Left ←
  - Right →
- **Reward (R)**
  - +100 → reaching goal
  - -1 → each step (to encourage shortest path)
  - -100 → hitting wall
- **Policy (π)**

- Strategy that tells the agent which action to take

Step-3: Choose Algorithm: Q-Learning

Q-Learning is commonly used for maze problems.

Q-Table:

A table storing:

- Q(state, action) → expected reward

Step-4: Q-Learning Formula

$$Q(s,a) = Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

Where:

- α = learning rate (e.g., 0.1)
- γ = discount factor (e.g., 0.9)
- r = reward
- s' = next state

Step-5: Step-by-Step Training Procedure

1: Initialize

- Create Q-table with all values = 0
- Define maze grid (0 = path, 1 = wall)

2: Training Loop

Repeat for many episodes (e.g., 1000):

For each episode:

1. Start at initial position (red arrow)
2. Repeat until goal reached:
    - Choose action:
        - **Exploration (ε-greedy)** → random move
        - **Exploitation** → best Q-value action
    - Take action → move to next state
    - Get reward:
        - Goal → +100
        - Wall → -100
        - Normal step → -1
    - Update Q-table using formula

3: Reduce Exploration

- Gradually reduce ε (epsilon)
- So agent becomes more greedy over time

Step-6: Extract Optimal Path

After training:

1. Start from initial position
2. At each step:
    - Choose action with **highest Q-value**
3. Continue until goal

Step-7: Output Path (Example Format)

Final learned path will look like:

- Start → Right → Right → Down → Down → Left → Down → Right → Goal

Or as coordinates:

- (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → ... → Goal